Robot Learning for Household Tasks: SO101 Demonstrations, OriginX Adaptation, and Training-Free Execution

Qinzhen Ma
MECE Program, Department of Electrical and Computer Engineering, Rice University
ELEC 594 Capstone, Fall 2026 — Midterm Report
qm18@rice.edu

***Abstract*—Household manipulation requires a robot to know which object to manipulate, which task stage it is in, and whether that stage is complete. We study these problems in two tracks. On a real SO101 arm, we recorded demonstrations of three household tasks and built a 102-episode cube-into-cup dataset in LeRobot format. In RoboCasa365 simulation, we adapted the Xiaomi-Robotics-1 vision-language-action policy with an Action LoRA and a 4.22M-parameter conditioning branch; an author-run evaluation reached 1,496/2,500 successes (59.84%), but a matched comparison showed no clear gain. We then built OriginX-TF, a training-free harness that adds object identity, stage contracts and visual verification around a frozen policy. It runs unchanged with two frozen policies and passes 145 CPU tests. A pilot (12 of 14 episodes completed) shows no success gain yet and points to identity recovery and stage verification as bottlenecks.**

I. Introduction

Robots that help in homes must handle rigid, deformable and articulated objects, and they must keep track of task progress over long horizons. Recent vision-language-action (VLA) policies map camera images, a language instruction and the robot state directly to actions [1]. When such a policy fails, the failure is not always a low-level motion error: the policy may act on the wrong object, repeat a finished stage, or move on before a stage is done. Language-model planners [2], [3] and recent harnesses around frozen VLA policies [4], [5] decompose a task and check its progress outside the policy.

This project follows two tracks (Fig. 1 illustrates the first and Fig. 2 part of the second). The *real-robot track* builds a data and training pipeline on a low-cost SO101 arm [6] with LeRobot [7]. The *simulation track* uses the RoboCasa365 kitchen benchmark [8], [9], where initial states can be controlled and closed-loop success is measured by the environment. At the midterm, the project has produced: (i) teleoperated demonstrations of three household tasks and a LeRobot dataset for one of them; (ii) an adapted Xiaomi-Robotics-1 policy (OriginX) with a complete 2,500-episode evaluation [10]; and (iii) OriginX-TF, a training-free execution harness that runs with two different frozen policies. The engineering results are verified; a task-success improvement is not yet shown, and we state this limit explicitly.

II. Methods

A. SO101 Demonstration Pipeline

The SO101 follower arm has five revolute joints and a gripper (six position-controlled actuators: shoulder pan, shoulder lift, elbow flex, wrist flex, wrist roll and gripper). Demonstrations are recorded by teleoperation with a leader arm. We recorded three tasks that cover different object types: placing a cube into a cup (rigid object), folding clothes (deformable object) and closing a toolbox (articulated object). The cube task is organized as a LeRobot v3 dataset in which each frame stores the six-dimensional joint state, the six-dimensional action, and one 640×480 RGB image from each of two cameras (front and wrist), at 30 Hz (Fig. 1). As a first policy baseline, we trained an Action Chunking Transformer (ACT) [11] with a chunk size of 100 for 30k steps on the 100 cube episodes available at the time.

B. OriginX: Adapting a Frozen VLA Policy

Xiaomi-Robotics-1 [1] combines a vision-language backbone with a flow-based action transformer. We start from its public RoboCasa365 checkpoint. In Stage A we adapt the action model with a low-rank adapter (Action LoRA) [12]. In Stage B we freeze the adapted policy and train a 4.22M-parameter conditioning branch that uses vision-language features and recent robot state to condition action generation. A stage-prediction objective is used during training only; it is not an input at test time.

C. OriginX-TF: A Training-Free Execution Harness

OriginX-TF moves stage management outside the policy and makes no parameter updates (Fig. 2). It has four parts.

Object identity. A frozen OWLv2 detector [13] proposes kitchen-object candidates, and a causal tracker uses frozen CLIP [14] appearance features to associate them over time, keeping up to 32 candidates with persistent track IDs. These IDs are visual handles, not simulator ground truth.

Stage contracts. Before the first action, a frozen Qwen3-VL-4B-Instruct model [15] reads the task, the three camera views and the candidates, and writes one to four contracts. Each contract has a stage instruction (e.g., "pick up the mug"), a target ID, and a visual postcondition for that stage only.

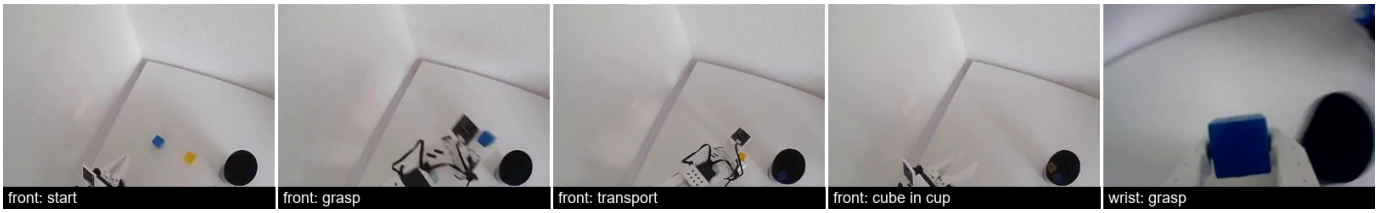


Fig. 1. One recorded SO101 demonstration (episode 0 of the 102-episode cube-into-cup dataset, 912 frames). Four front-camera frames from start to placement and one wrist-camera frame at the grasp. Clothes folding and toolbox closing were also recorded on SO101; they are not included in these dataset statistics.

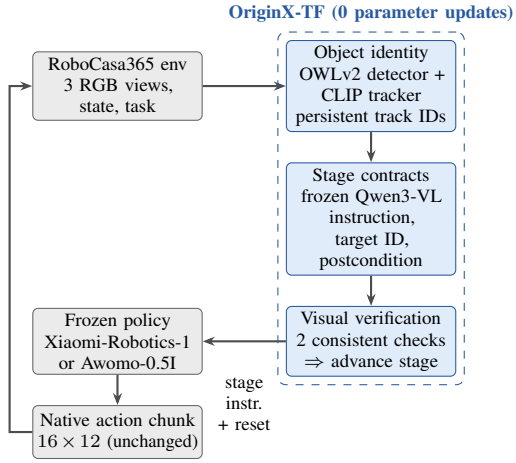


Fig. 2. OriginX-TF wraps a frozen policy. Blue blocks are the harness; gray blocks are unchanged components. All models are frozen.

Visual verification. Every 64 control steps, the same frozen model compares the three stage-reference views with the three current views and checks the bound target and the postcondition. The state machine advances only after two consistent confirmations at different times.

Portable policy interface. The harness reads the public observations (three RGB views, proprioception and the instruction), passes the current stage instruction as natural language, and resets the policy's history when the instruction changes. The policy's native action chunk (16 steps × 12 action dimensions) is executed without modification. If the bound target is not visible, the harness falls back to the original task instruction. Task success is always decided by the environment's evaluator, not by the harness. The same harness is connected to Xiaomi-Robotics-1 and to Awomo-0.5I [16], each with its own frozen checkpoint.

D. Evaluation Protocol

For the earlier adaptation study we ran 50 RoboCasa365 tasks with 50 episodes each. For OriginX-TF we ran a small fixed pilot on two tasks, CoffeeSetupMug (coffee) and PickPlaceCounterToCabinet (cabinet), with one seed per task and seven experimental arms: native Xiaomi, Xiaomi with the full harness, three Xiaomi ablations (*no-stage*: no stage instructions or their history resets; *no-binding*: no identity/visibility gating; *single-confirm*: one confirmation instead of two), native Awomo, and Awomo with the full harness. This gives 14 planned episodes.

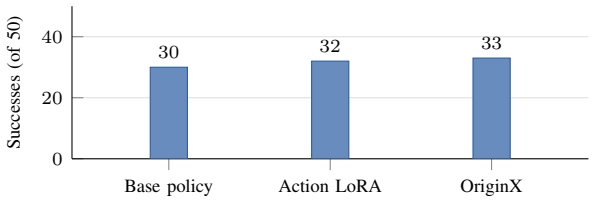


Fig. 3. Separate 50-case RoboCasa365 comparison of the adaptation study. On 44 matched LoRA/OriginX pairs: 5 wins, 4 losses, 35 ties (6 unresolved).

Comparisons are made only between episodes with matching initial-state fingerprints, and episodes without a terminal result stay in the denominator as *unknown*. All harness runs used a single RTX 5090 Laptop GPU with 24 GB of memory, loading the models sequentially.

III. Results (Partial)

A. Real-Robot Data

The cube-into-cup dataset contains 102 episodes and 65,620 frames (mean 643 frames, about 21 s per episode) with front and wrist video (Fig. 1). ACT training finished all 30k steps; its closed-loop success on the real arm is not yet reported.

B. OriginX Adaptation in RoboCasa365

The full author-run evaluation of the stage-supervised model finished all 2,500 episodes with 1,496 successes (59.84%). This is our own evaluation, not an official leaderboard entry. To test whether the added branch helps, we ran a separate 50-case comparison (Fig. 3). Raw successes were 30/50 for the base policy, 32/50 for Action LoRA and 33/50 for OriginX. Among 44 matched LoRA/OriginX pairs, OriginX won 5, lost 4 and tied 35; 6 comparisons are unresolved. The added benefit of the branch is therefore uncertain.

Two failure studies motivated the harness. Supplying one language subgoal from gpt-6-astra (high reasoning effort, given the three current views) to the frozen OriginX policy produced 56 verified rescues (failures turned into successes) among 1,004 historical failures [10]. In a visual-grounding pilot with the frozen base Xiaomi policy, an oracle-placed (correct) visual cue versus the raw input gave 3 wins and 1 loss over 45 matched cases, with 3 of 48 planned cases unresolved.

C. OriginX-TF Engineering Checks

The same harness, with identity-checked source code, perception front end, prompts and configuration, runs closed-loop

	Coffee	Cabinet
Xiaomi native	S*	F
Xiaomi + TF	S*	F
TF no-stage	S*	F
TF no-binding	F*	S (256)
TF single-confirm	unknown	unknown
Awomo native	S	F
Awomo + TF	S	F

Fig. 4. OriginX-TF (TF) pilot outcomes, one seed per task. S = success, F = failure; S (256) = success at step 256. * Coffee pairs with Xiaomi + TF are excluded from paired inference because its initial state does not match the other Xiaomi coffee episodes. The full harness failed the cabinet task at the 750-step limit.

RoboCasa episodes with both frozen policies and makes zero parameter updates. The test suite passes 145 CPU tests, with 2 skipped because of Windows symbolic-link permissions. For both backends, replaying the same static observations with and without moving the policy weights between CPU and GPU gave identical native actions, which supports the single-GPU sequential loading. These checks establish engineering feasibility and cross-backend reuse; they do not measure task success.

D. OriginX-TF Pilot

Twelve of the 14 planned episodes finished: 6 successes and 6 task failures; the other 2 are unknown (Fig. 4). These counts mix baselines and ablations and are not a success rate of the method. Xiaomi solved 1 of 2 tasks with or without the harness, and so did Awomo. Because the Xiaomi coffee-task episodes do not share a matched initial state with the full-harness episode, only three main pairs are valid, and all three show no difference. The no-stage and no-binding ablations each solved 1 of 2 tasks; both single-confirm episodes are unknown. In the cabinet task, where initial states match, the no-binding ablation succeeded at step 256 while the full harness failed at the 750-step limit. None of the four full-harness episodes committed a stage transition, and neither did the successful no-binding episode.

IV. Discussion (Partial)

The midterm results separate what works from what is not yet shown. The data pipeline, the adapted policy and its large evaluation, and a harness that runs unchanged on two frozen policies on a single laptop GPU are complete. A task-success gain is not established: the adaptation study shows a small, uncertain difference, and the harness pilot shows no difference on its three valid main pairs.

The pilot logs point to a concrete bottleneck. Perception updates every 16 control steps, while a track expires after 30 steps without a match, so a single missed detection expires it; once the initial target ID expires, the harness has no path to rebind it and keeps falling back to the original instruction. Some verifier answers also mix the current stage with the final goal (for example, judging a grasp stage by the final placement). This is consistent with the absence of stage commits and with the matched cabinet case, where removing the identity gate (no-binding) succeeded even without a stage commit. One case, however, cannot establish a general effect in either direction.

The next version will (1) recover object identity after occlusion by distinguishing temporary invisibility, re-identification and target replacement; (2) ask the verifier only about the current postcondition and separate "not done" from "uncertain"; and (3) record full initial-state components so that unmatched pairs can be diagnosed. We will first finish the 2 unknown pilot episodes with the unchanged version, then develop on the current cases, freeze the configuration, and evaluate on fresh held-out cases, reporting success rate, wrong-target commits, recoveries and inference cost. On the real robot, we will extend the folding and toolbox recordings and measure closed-loop success of the trained policies on the SO101 arm.

Acknowledgements

We thank the ELEC 594 instructors for feedback on weekly reports, and H. Yang, J. Wu, S. Tang and G. Dai, co-authors of the OriginX study [10]. AI assistants (OpenAI Codex and Anthropic Claude) were used for coding support, experiment orchestration, log analysis and drafting of this report; the author reviewed the content and is responsible for it. Code and evidence for the adaptation study are available at https://github.com/Quinn-Ma/OriginX.

CRediT Author Statement

Qinzhen Ma: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing.

References

[1] Xiaomi Robotics Team, "Xiaomi-Robotics-1: Scaling vision-language-action models with over 100K hours of real-world trajectories," arXiv:2607.15330, 2026.

[2] B. Ichter, A. Brohan, Y. Chebotar *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. 6th Conf. Robot Learning (CoRL), PMLR 205*, 2023, pp. 287–318.

[3] W. Huang, F. Xia, T. Xiao *et al.*, "Inner monologue: Embodied reasoning through planning with language models," in *Proc. Conf. Robot Learning (CoRL)*, 2022.

[4] Z. Li, Z. Li, K. Zhou, and J. Gu, "RoboHarness: A memory-augmented policy harness for vision-language-action model robustness via in-context adaptation," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2026, arXiv:2603.24060.

[5] H. Gao, Z. Ni, X. Wen, S. Xu, R. Li, R. Xu, and Y. Wang, "AGM: Achievement-grounded memory for closed-loop agents with frozen VLA policies," arXiv:2608.29537, 2026.

[6] R. Knight, P. Kooijmans, R. Cadene, S. Alibert *et al.*, "Standard open SO-100 & SO-101 arms," https://github.com/TheRobotStudio/SO-ARM100, 2024.

[7] R. Cadene, S. Alibert, A. Soare *et al.*, "LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch," https://github.com/huggingface/lerobot, 2024.

[8] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu, "RoboCasa: Large-scale simulation of everyday tasks for generalist robots," in *Proc. Robotics: Science and Systems (RSS)*, 2024.

[9] S. Nasiriany, S. Nasiriany, A. Maddukuri, and Y. Zhu, "RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026, code: https://github.com/robocasa/robocasa.

[10] Q. Ma, H. Yang, J. Wu, S. Tang, and G. Dai, "OriginX: Continuous conditioning of a frozen robot policy for RoboCasa365," Technical report, https://github.com/Quinn-Ma/OriginX, 2026.
[11] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proc. Robotics: Science and Systems (RSS)*, 2023.
[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
[13] M. Minderer, A. Gritsenko, and N. Houlsby, "Scaling open-vocabulary object detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
[14] A. Radford, J. W. Kim, C. Hallacy *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Machine Learning (ICML)*, 2021.
[15] Qwen Team, "Qwen3-VL-4B-Instruct," Model card, https://huggingface.co/Qwen/Qwen3-VL-4B-Instruct, 2025.
[16] Awomo-WAM Team, "Awomo-0.5," Software release, https://github.com/Awomo-WestlakeDI/Awomo-0.5, 2026.